NNDL project: ResNet-Based Approaches to Coarse and Fine-Grained Car Recognition

Marco Cavazza†

***Abstract*—This project, developed as the final work for the course *Neural Networks and Deep Learning* at the University of Padua, applies convolutional neural networks to car image classification using the CompCars dataset. The goal is not to introduce methodological innovations, but to consolidate the theoretical foundations acquired during the course and demonstrate their application to a realistic case study.**

We implemented and tested several ResNet-based architectures, trained both from scratch and with pretrained weights. In the task of car make recognition, the from-scratch network achieved accuracies above 90%, confirming the correct implementation and the effectiveness of the learning techniques. Pretrained model consistently outperformed those trained from scratch, highlighting the importance of leveraging prior knowledge when data are limited.

In the more challenging setting of fine-grained classification of 50 BMW models, we experimented the Focal Loss to address class imbalance. However, the results showed that even with these techniques, performance across classes remained uneven due to the limited dataset size and the strong visual similarity among models.

Overall, the work demonstrates both the potential of deep neural networks for automotive classification and the difficulties that arise in fine-grained, imbalanced scenarios, providing a pedagogical reference for students and practitioners approaching deep learning in practice.

***Index Terms*—Deep Learning, Convolutional Neural Networks, Transfer Learning, Image Classification, Automotive Vision**

I. Introduction

Cars are a challenging object class for computer vision: many brands and models differ only in subtle details, while real-world images add variability from viewpoint, scale, lighting, and occlusion.

In this report, we compare different neural network approaches, combining modern techniques studied in the Neural Networks and Deep Learning course with standard models from the `torchvision.models` library. Experiments are conducted on a small subset of CompCars dataset [1], which, despite its age, remains a diverse benchmark. With its wide range of car types and viewpoints, CompCars provides a suitable testbed for studying the challenges of training and evaluating deep learning models for vehicle recognition.

II. Related Work

Early vehicle recognition relied on handcrafted features and geometric models. For example, Zhang et al. [2] proposed a 3D deformable model for monocular images, while Hsiao et al. (2014) [3] and Lin et al. (2014) [4] introduced curve-based and joint 3D fitting strategies to better handle pose and shape variability. These approaches laid important foundations but required heavy manual design and often struggled to adapt to real-world data.

With the advent of deep learning, end-to-end convolutional neural networks (CNNs) replaced handcrafted pipelines by learning directly from raw images. Models trained on CompCars achieved strong performance without manual preprocessing. Among modern architectures, ResNet [5] was particularly influential: its residual connections enabled much deeper networks and established it as a standard backbone for fine-grained recognition, including vehicle make and model classification. Building on this success, in this project we focus on ResNet-based architectures as the starting point for our experiments.

III. Processing Pipeline

The work developed in this project follows a sequence of five main steps, from the preparation of the data to the evaluation of the trained models (Fig. 1):

1) **Dataset.** For the experiments we relied on a reduced version of the CompCars dataset as explained in the next section. In the first case, a subset was chosen to study the recognition of different car manufacturers. In the second case, the analysis was conducted on BMW images only, in order to address a more fine-grained task. Here the goal was to distinguish among the different models of a single brand.
2) **Preprocessing and Data Augmentation.** All images were resized and cropped before being passed to the network according to the BBS (Bounding Boxes) reproted in the label files. To improve generalization, several forms of augmentation were applied: random rotations, horizontal flips, color variations, Gaussian blur, and standard normalization. The dataset was split into training, validation, and test sets.
3) **Models.** We implemented three CNNs with increasing complexity:
   - **Baseline CNN**: a less deep architecture used as a starting point.
   - **ResNet18**: residual network plus a decision head.
   - **ResNet50**: deeper residual network plus decision head.

   The three models were first trained from scratch on our data. Then we tested a *pretrained* version of the best-

†Department of Physics and Astronomy of Padova,
email: marco.cavazza.1@studenti.unipd.it

performing architecture (**ResNet18** in our experiments) and compared with to the from-scratch setup.

4) **Training.** Training was carried out using the Adam optimizer, combined with a cosine annealing scheduler to adapt the learning rate across epochs. Different loss functions were evaluated: the standard `CrossEntropyLoss` and the `FocalLoss`. During training, the model state corresponding to the lowest validation loss was saved and later used for evaluation.
5) **Evaluation.** The assessment of the models included global metrics such as accuracy, precision, recall, macro-F1 and ROC-AUC. To gain a deeper understanding of the results, per-class accuracy and F1-scores were also reported, together with confusion matrices and plots of the class distribution.

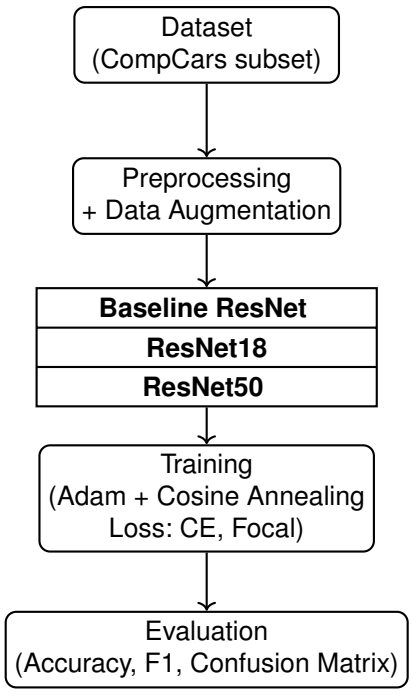

Fig. 1: Workflow of the project: from dataset preparation to evaluation, with three models of increasing complexity.

IV. Signals and Features

The experiments in this project are based on the CompCars dataset, a large-scale benchmark designed for fine-grained vehicle recognition and verification. The dataset contains over 200,000 images collected from two scenarios: *web-nature* (car forums, websites, and search engines) and *surveillance-nature* (traffic cameras). This dual-source structure introduces both clean, high-resolution images and more challenging real-world samples affected by lighting, weather, occlusion, and varying viewpoints.

The dataset is organized as follows:

- **image/** stores full-car images in the format `make_id/model_id/released_year/image_name.jpg`.
- **label/** provides annotation files including viewpoint information (front, rear, side, front-side, rear-side), bounding box coordinates, and the number of detected objects.
- **part/** contains cropped images of eight specific car parts (headlight, taillight, fog light, air intake, console, steering wheel, dashboard, and gear lever).
- **misc/** includes attribute files such as maximum speed, displacement, door and seat numbers, and car type (12 predefined categories, e.g., sedan, SUV, hatchback).
- **train_test_split/** offers predefined splits for classification, part recognition, and verification tasks, including easy/medium/hard verification pairs (it wasn not used during in this project).

The images are labeled with a three-level hierarchy (Fig. 2), allowing both coarse-grained and fine-grained classification. For example, an Audi A4L appears across different years and trims, reflecting subtle intra-class variations.

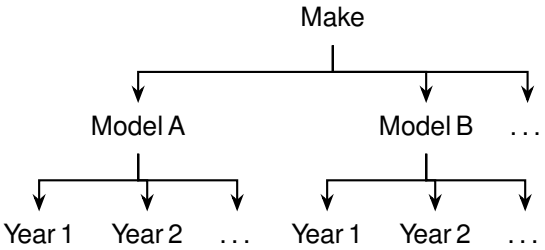

Fig. 2: Hierarchical organization of the CompCars dataset: make → model → year.

In our experiments, we tried to use a balanced subset of CompCars to ensure an even distribution across classes (it was not perfectly balanced though), which is important for fair evaluation under limited resources (Fig. 3). In a second stage, we restricted the dataset to vehicles from the **BMW brand**, focusing on distinguishing between different BMW models (Fig. 4). This second case allowed us to first study the effect of class unbalance balance and then to test fine-grained recognition within a single manufacturer, where inter-class differences are often subtle.

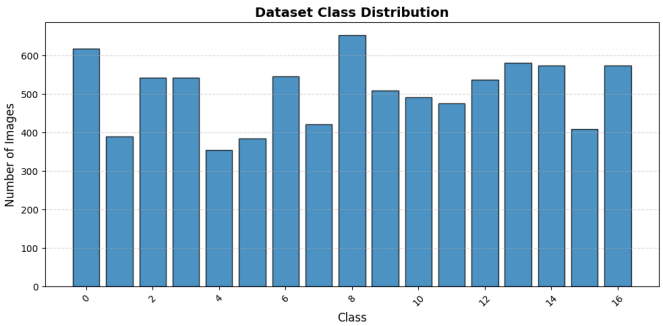

Fig. 3: Distribution of the classes corresponding to different car makes.

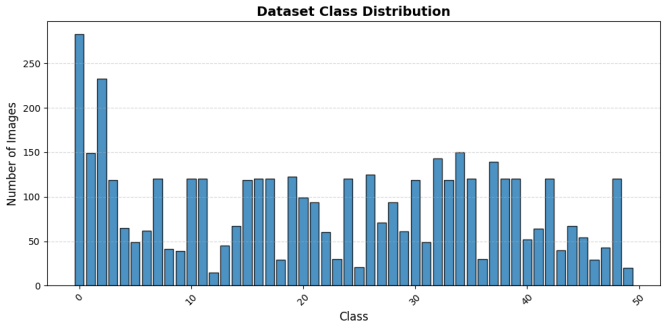

Fig. 4: Distribution of the 50 classes corresponding to BMW models.

Although the original data set release provides predefined train/test splits, these were not adopted in our experiments. Due to practical limitations such as data transfer from cloud storage and limited computational power, we opted to construct our own subsets tailored to the scope of this work.

Overall, the original dataset provides a rich variety of viewpoints, attributes, and hierarchical labels, making it an excellent basis for exploring both classical and deep learning approaches in vehicle recognition, therefore, we encourage to use a larger portion of the dataset if you setup allows it.

V. Learning framework

A. Models

In this work three residual networks of increasing complexity were implemented and compared (Figure 7). Each architecture follows the ResNet paradigm but differs in depth and structural choices [5]. Residual Networks (ResNets) address the gradient degradation problem in deep learning by introducing *skip connections*: instead of directly approximating a mapping $\mathcal{H}(x)$, the stacked layers learn a residual function $\mathcal{F}(x) := \mathcal{H}(x) - x$. The final output is then expressed as

$$y = \mathcal{F}(x) + x, \tag{1}$$

which allows gradients to propagate more effectively and enables the training of much deeper architectures.

The first model we see, referred to as **ResNet_B**, is our baseline. It begins with a convolutional stem (7×7 Conv, BatchNorm, ReLU, MaxPool) and then four `BasicBlocks` like the one showed in Fig. 5. A block contains two 3×3 convolutions with BatchNorm and ReLU, plus a skip connection, which can be identity or a Conv1×1 when dimensions change. The network ends with global average pooling and a fully connected layer. Counting only convolutional and fully connected layers, the model has 13 trainable layers.

The second architecture is a **ResNet18** extended with a custom decision head. Compared to ResNet_B, it has double the `BasicBlocks`, resulting in a deeper backbone. After global average pooling, instead of the standard linear classifier, we added a two-layer head with BatchNorm, ReLU, and Dropout. This modification improves regularization and capabilities of the model.

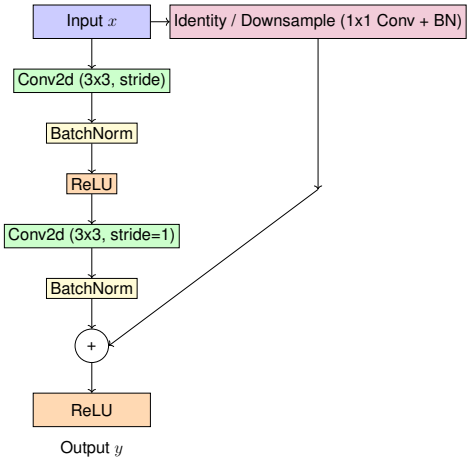

Fig. 5: Structure of a BasicBlock: two 3x3 convolutions with BatchNorm, ReLU, and a skip connection (identity or downsample).

The third model is a modified **ResNet50**. There are two possible basic blocks that constitute the backbone of the structure. Each block has three convolutions (1×1, 3×3, 1×1). `IdentityBlock` modules preserve input dimensions, while `ConvolutionalBlock` modules perform downsampling with stride 2 and an then follows the shortcut path (Fig. 6).

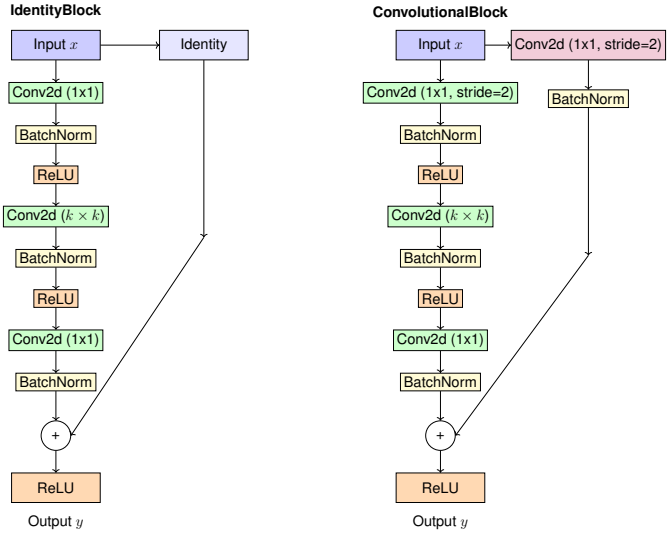

Fig. 6: Comparison of residual blocks: IdentityBlock (left) and ConvolutionalBlock (right).

Once the build blocks are established we organize the architecture into four stages:

- 1 ConvBlock + 2 IdentityBlocks,
- 1 ConvBlock + 3 IdentityBlocks,
- 1 ConvBlock + 5 IdentityBlocks,
- 1 ConvBlock + 2 IdentityBlocks.

In summary, the three implementations form a clear progression: ResNet_B as a compact baseline, ResNet18 as a medium-depth model with an enhanced head, and ResNet50

as a deep architecture. This setup enabled a systematic comparison of accuracy, generalization, and computational cost across different model complexities.

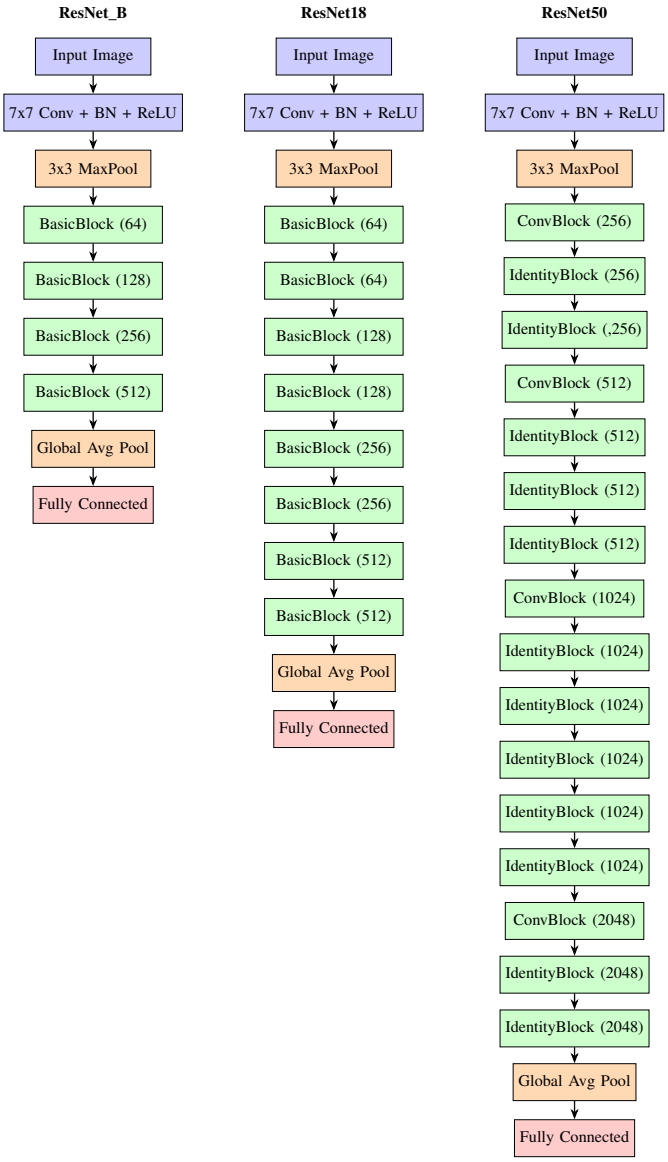

Fig. 7: Architectures: ResNet_B (left), ResNet18 (center), and ResNet50 (right).

B. Hyperparameters and Training Setup

During training, several hyperparameters and design choices were made to ensure both robustness and efficiency.

a) Data augmentation: To improve generalization and reduce overfitting, we applied standard image augmentation strategies. All images were resized to $(256, 256)$ and then randomly cropped to $(224, 224)$. Additional transformations included random horizontal flips, small rotations, color jitter, and occasional Gaussian blur. This ensured that the model saw slightly different versions of the same car, capturing variability in viewpoint and lighting.

b) Batch size: A batch size of 64 was chosen. Given the relatively limited dataset, this value allowed us to maintain useful stochasticity in gradient updates, helping the optimizer escape poor local minima without excessively increasing memory usage.

c) Optimizer: We used Stochastic Gradient Descent (SGD) with weight decay (L2 regularization), a classical yet reliable choice for deep convolutional networks.

d) Learning rate scheduling.: A cosine annealing scheduler was employed to gradually reduce the learning rate over time. Specifically, we used `CosineAnnealingLR` with $T_{\text{max}} = 60$ (the total number of epochs in a full cosine cycle) and $\eta_{\text{min}} = 10^{-3}$ as the minimum learning rate. This smooth decay schedule avoids sudden drops and helps the optimizer converge to more stable minima.

e) Loss functions: The primary loss was the categorical cross-entropy. To address class imbalance and focus learning on hard examples, we also experimented with the focal loss, defined as:

$$\mathcal{L}_{\text{focal}}(p_t) = -\alpha(1 - p_t)^{\gamma} \log(p_t), \tag{2}$$

where p_t is the predicted probability of the true class, α is a weighting factor for class imbalance, and γ modulates the focus on difficult samples.

f) Training loop: Models were trained for 60 epochs. Empirical analysis showed that beyond this point validation performance plateaued, with negligible gains. Therefore, to balance computational cost with performance, training was limited to 60 epochs. Each epoch included a complete pass through the training set with loss and accuracy monitored on the validation set.

g) Evaluation Metrics: During training and evaluation, we monitored several global and per-class metrics, together with visual analysis tools. These metrics provide complementary insights: global metrics summarize overall performance, while per-class metrics highlight potential imbalances across categories.

1) **Loss**
   Numerical value of the loss function used for optimization.
2) **Accuracy**
   Overall percentage of correct predictions:
   $$\text{Accuracy} = \frac{\#\text{ correct predictions}}{\#\text{ total samples}} \tag{3}$$
3) **Precision (macro)**
   Average precision across all classes:
   $$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i} \tag{4}$$
   where TP_i are True Positives, FP_i False Positives, and FN_i False Negatives. The *macro* version computes the arithmetic mean across all classes.
4) **Recall (macro)**

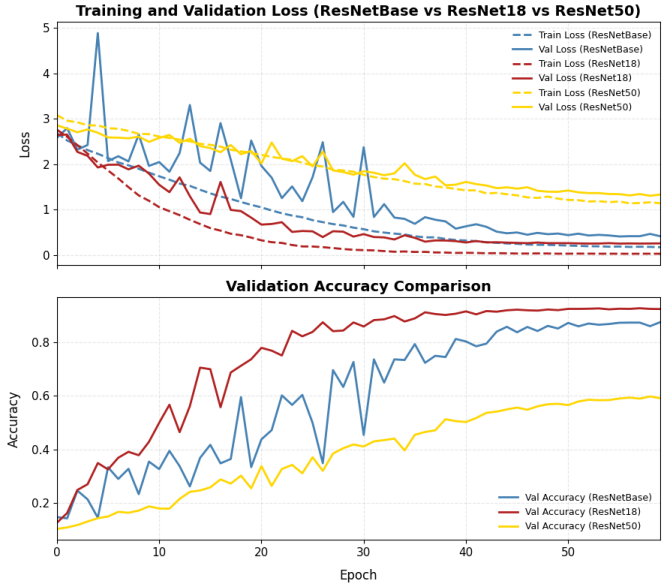

Fig. 8: Caption

Average recall across all classes:

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i} \tag{5}$$

5) **F1-score (macro)**
Harmonic mean of precision and recall, averaged across classes:

$$F1_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \tag{6}$$

6) **ROC-AUC (multi-class, OvR)**
Area under the ROC curve, extended to multiple classes using the one-vs-rest (OvR) strategy. For each class, the ROC curve is computed by treating that class as positive and all others as negative; the final score is the average across all classes. This provides a global measure of the model's ability to discriminate each class against the rest.

VI. Results

We first present the results obtained by training the three models — ResNet_B, ResNet18, and ResNet50 — from scratch on a reduced subset of the CompCars dataset. Figure 8 shows the comparison of the training and validation losses, as well as the validation accuracy across epochs.

From the curves we observe that the most performant model in this setting is ResNet18, which achieves the highest validation accuracy, approaching 90%. This is consistent with the size of the training set: given the limited amount of data, simpler models such as ResNet_B and ResNet18 are sufficient to generalize well and reach high accuracy levels.

In contrast, ResNet50, despite being potentially the most powerful model, struggles to adapt in this low-data regime. Both its validation accuracy and its other evaluation metrics remain consistently below those of the simpler models (Tab. 1), indicating that its higher capacity is not fully exploited.

TABLE 1: Metrics for different models in car make classification.

Model	Loss	Accuracy	Precision	Recall	F1-score	ROC-AUC
ResNet18	0.2297	0.9338	0.9373	0.9352	0.9357	0.9979
ResNet50	1.3308	0.5918	0.5918	0.5918	0.5918	0.5918
ResNet_Base	0.3804	0.8943	0.8946	0.8970	0.8950	0.9945
ResNet18 (Pretrained)	0.0745	0.9785	0.9779	0.9786	0.9782	0.9998

With limited data, most of its additional parameters remain underused, while the extra depth adds unnecessary complexity.

We continue the analysis of the first numerical experiments by examining the confusion matrix of the best-performing model, ResNet18 (Figure 9). Together with the per-class accuracy and F1-score, the results show a fairly uniform distribution across all classes, without significant drops in performance. This highlights the model's ability to generalize well and maintain balanced recognition among categories.

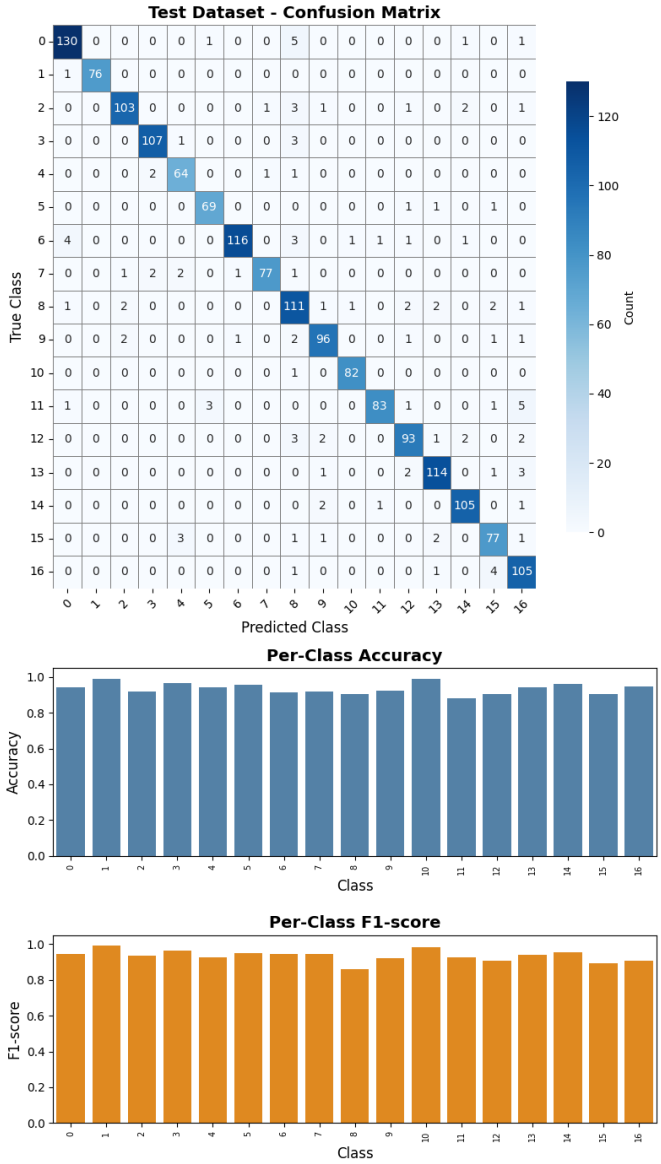

Fig. 9: Confusion matrix, per-class accuracy, and per-class F1-score for the best-performing model in the car make problem.

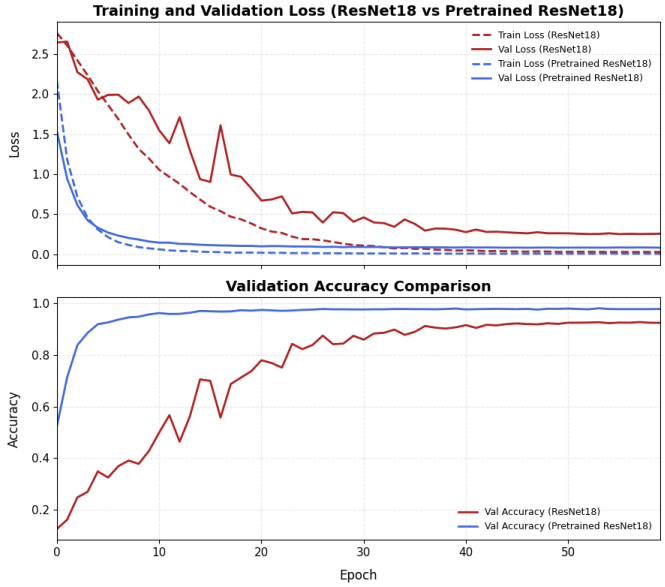

Fig. 10: Comparison between ResNet18 trained from scratch and pretrained on ImageNet (car make classification). The pretrained model not only achieves higher final performance but also converges faster, reaching its peak accuracy and stable validation loss earlier than the scratch model.

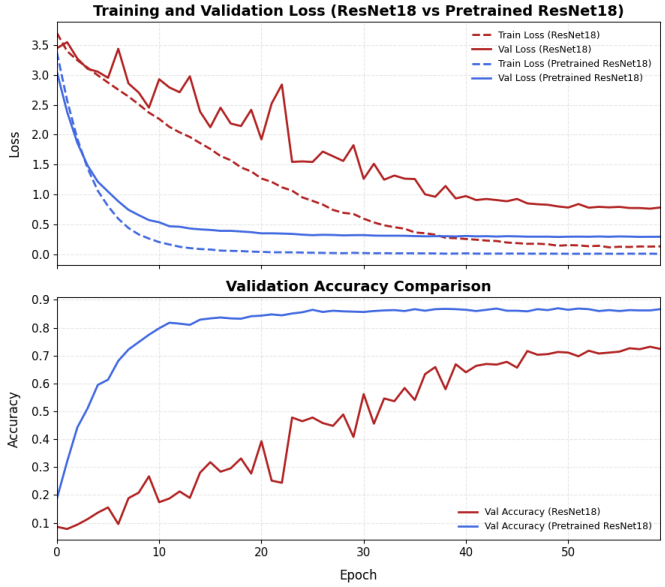

Fig. 11: Training and validation curves for ResNet18 trained from scratch and with pretrained weights for the BMW models classification problem.

Figure 10 compares the performance of a ResNet18 pretrained on ImageNet with the same architecture trained from scratch on our subset. The pretrained model clearly outperforms the scratch version across all metrics. This advantage arises because pretraining provides strong feature representations learned from millions of natural images, which transfer well even to specific tasks such as car recognition. In contrast, training from scratch with limited data leads to weaker, dough valid, generalization.

What showed in the graph is confirmed in Table 1, demonstrating the clear benefit of leveraging transfer learning in data-limited scenarios.

In the final part of our analysis, we addressed the challenge of fine-grained classification by focusing on the dataset folder containing all BMW models (50 classes, ∼5000 images). Since the class distribution was highly unbalanced (Fig. 4), we employed a modified Focal Loss (Eq. 2) combined with class weighting, defined as:

$$\texttt{class_weights} = \frac{1}{\sqrt{n_i}} \Big/ \frac{1}{C}\sum_{j=1}^{C}\frac{1}{\sqrt{n_j}}, \tag{7}$$

This formulation gives more weight to underrepresented classes while avoiding extreme imbalance.

We compared ResNet18 trained from scratch with its pretrained counterpart (Fig. 11). Results were not encouraging: even with Focal Loss and class weighting, the per-class metrics (accuracy and F1-score) remained highly uneven. The pretrained model achieved slightly higher homogeneity (Tab. 2), but still failed to classify all classes consistentl (Fig. 12).

Several factors likely explain this outcome. First, the dataset was small, leaving many classes with too few examples. Second, many BMW models are visually very similar, making the classification task inherently difficult. Third, while Focal Loss highlights hard examples, it does not guarantee balanced performance when classes are extremely rare; in some cases, it may even over-emphasize noise or outliers. Finally, deeper or shallower networks did not improve results, likely due to overfitting or insufficient capacity to separate subtle inter-class differences.

TABLE 2: Metrics for different models in model classification.

Model	Loss	Accuracy	Precision	Recall	F1-score	ROC-AUC
ResNet50	2.2008	0.3297	0.3836	0.3243	0.3151	0.8936
ResNet18	0.8065	0.7266	0.7561	0.7309	0.7338	0.9811
Base ResNet	1.2872	0.5590	0.5922	0.5522	0.5497	0.9560
ResNet18 (Pretrained)	0.2839	0.8644	0.8614	0.8648	0.8598	0.9971

TABLE 3: Test set performance of different ResNet variants. Pretrained ResNet18 shows the best overall results, while models trained from scratch achieve lower accuracy and stability.

In short, the lack of uniformity appears to be less a limitation of the loss function itself and more a consequence of dataset size and class similarity.

VII. Concluding Remarks

In this project, convolutional networks were applied to the CompCars dataset for both car make recognition and fine-grained BMW model classification. ResNet18 proved to be the best compromise between model complexity and the available data, reaching over 90% accuracy in make recognition, while pretrained models provided an extra boost in performance.

The fine-grained BMW task emphasized the difficulty of distinguishing very similar classes, where imbalance and subtle differences limited the benefit of methods such as Focal

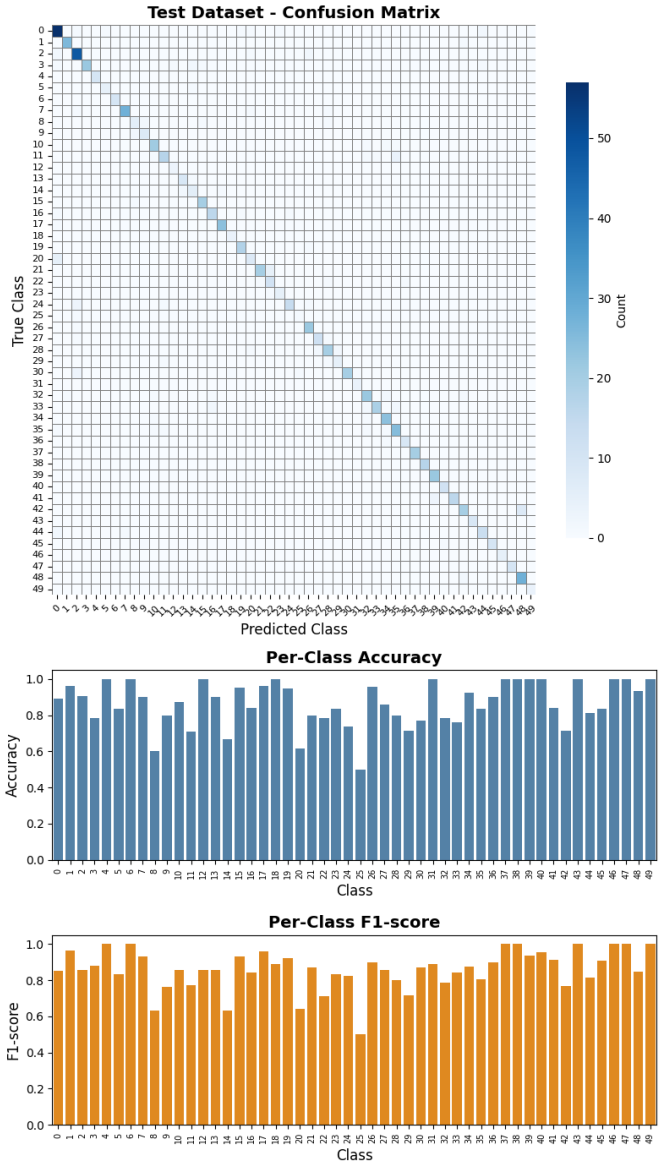

Fig. 12: Confusion matrix, per-class accuracy, and per-class F1-score for the pretrained ResNet18 on BMW model recognition. While the overall accuracy is high, performance remains uneven across classes.

Loss. Looking ahead, progress will likely depend on larger and more balanced datasets, or on techniques like metric and contrastive learning to better capture fine-grained differences.

On a personal note, this project not only helped me strengthen both theory and practice, but also gave me hands-on experience with tools that were entirely new to me. Coming from a background in theoretical physics, it was my first real opportunity to work with neural networks and to use object-oriented programming in Python, including writing my first classes. Beyond the technical results, it became an important step in building confidence with deep learning workflows.

References

[1] L. Yang, P. Luo, C. C. Loy, and X. Tang, "A large-scale car dataset for fine-grained categorization and verification," 2015.

[2] Z. Zhang, T. Tan, K. Huang, and Y. Wang, "Three-dimensional deformable-model-based localization and recognition of road vehicles," *IEEE Transactions on Image Processing*, vol. 21, no. 1, pp. 1–13, 2012.

[3] E. Hsiao, S. Sinha, K. Ramnath, S. Baker, L. Zitnick, and R. Szeliski, "Car make and model recognition using 3d curve alignment," pp. 1–1, 03 2014.

[4] Y.-L. Lin, V. Morariu, W. Hsu, and L. Davis, "Jointly optimizing 3d model fitting and fine-grained classification," vol. 8692, pp. 466–480, 09 2014.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015.